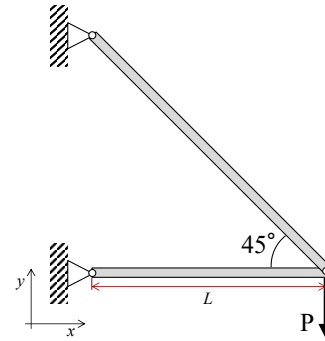


Basic FEM for trusses (Day 1)

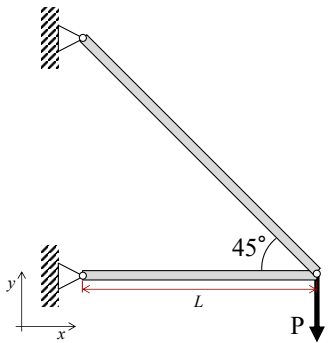
Ole Sigmund and Casper Schousboe Andreasen
Department of Civil and Mechanical Engineering, Solid Mechanics

Preparatory reading:
Handout Day 1
Chapter 1 in Cook et al.

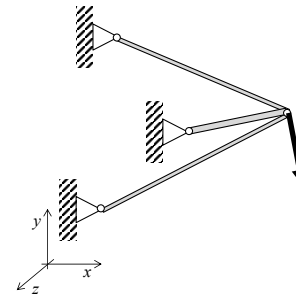
Simple truss structures (brush-up)



Simple truss structures (brush-up)



Simple truss structures (brush-up)



Virtual Work Principle

$$\int_V \{\delta \epsilon\}^T \{\sigma\} dV = \int_S \{\delta \mathbf{u}\}^T \{F\} dS + \int_V \{\delta \mathbf{u}\}^T \{\Phi\} dV + \sum_i \{\delta \mathbf{u}\}_i^T \{p\}_i,$$

which has to hold for any kinematically admissible displacement variations $\{\delta \mathbf{u}\}$, associated strain variations $\{\delta \epsilon\}$ and nodal displacement variations $\{\delta D\}$.

Or

$$\delta \Omega = \delta W$$

where

V is structural volume
 S is surface area
 $\{F\}$ is surface traction
 $\{\Phi\}$ is body force
 $\{\delta \mathbf{u}\}$ is virtual displacement variation
 $\{\sigma\}$ is the stress vector
 $\{\delta \epsilon\}$ is the virtual strain variation vector
 $\{p\}_i$ is a concentrated (nodal) load vector

Definition:

$$\delta \Omega \equiv \frac{\partial \Omega}{\partial D_i} \delta D_i$$

Virtual Work Principle (in words)

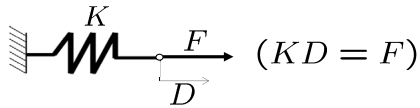
Or in words:

For a solution to be in equilibrium:

Arbitrary virtual displacement variations must lead to identical variations in internal and external work

i.e. structural equilibrium is an energy balance between internal and external work

VWP – 1 dof spring



Elastic (strain) energy:

$$\Omega = \frac{1}{2} K D^2$$

Work of external forces:

$$W = F D$$

$$\delta \Omega = \delta W \quad \forall \delta D (\neq 0)$$

$$\delta \Omega \equiv \frac{\partial \Omega}{\partial D_i} \delta D_i$$

Virtual Work Principle (truss analysis)

$$\int_V \{\delta \epsilon\}^T \{\sigma\} dV = \int_S \{\delta \mathbf{u}\}^T \{F\} dS + \int_V \{\delta \mathbf{u}\}^T \{\Phi\} dV + \sum_i \{\delta \mathbf{u}\}_i^T \{p\}_i,$$

Simplifications:

LHS: Uniaxial and constant stress/strain in bar elements

$$\{\delta \epsilon\}^T \{\sigma\} \rightarrow \delta \epsilon \sigma \text{ and } \int_V \rightarrow \sum_e \int_{V_e} \rightarrow \sum_e A^e L_0^e$$

RHS: No distributed forces

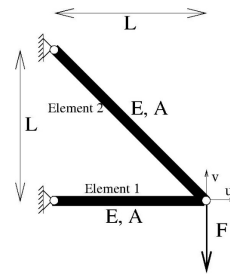
$$\{F\} = \{0\}$$

RHS: No volume loads

$$\{\Phi\} = \{0\}$$

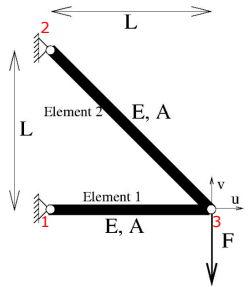
RHS: Only nodal loads

$$\sum_i \{\delta \mathbf{u}\}_i^T \{p\}_i = \{\delta D\}^T \{P\}$$



Virtual Work Principle (truss analysis)

$$\sum_e A^e L_0^e \delta \varepsilon = \{\delta D\}^T \{P\} \quad \forall \{\delta D\} \neq \{0\}$$



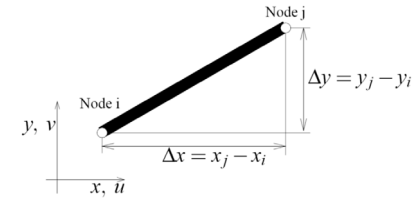
$$\{D\} = \{u_1 \ v_1 \ u_2 \ v_2 \ \dots \ u_N \ v_N\}^T$$

$$\{d^1\} = \{u_1 \ v_1 \ u_3 \ v_3\}^T$$

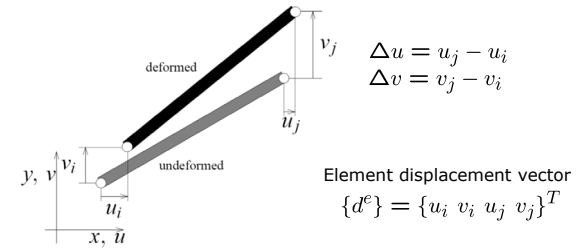
$$\{d^2\} = \{u_2 \ v_2 \ u_3 \ v_3\}^T$$

Truss element

Element geometry:



Deformed element:



Truss element

Initial and deformed length:

$$L_0 = \sqrt{\Delta x^2 + \Delta y^2}$$

$$\begin{aligned} L_1 &= \sqrt{(\Delta x + \Delta u)^2 + (\Delta y + \Delta v)^2} \\ &= \sqrt{\Delta x^2 + \Delta y^2 + \Delta u^2 + \Delta v^2 + 2(\Delta x \Delta u + \Delta y \Delta v)} \\ &\approx L_0 + \frac{\Delta x \Delta u + \Delta y \Delta v}{L_0} \end{aligned}$$

From 1st order Taylor-expansion assuming small displacements ($\Delta u \ll \Delta x$)

Elongation:

$$\Delta L = L_1 - L_0 = \frac{\Delta x \Delta u + \Delta y \Delta v}{L_0}$$

Truss element

Relative length change, i.e. strain:

$$\varepsilon = \frac{\Delta L}{L_0} = \frac{L_1 - L_0}{L_0} = \frac{\Delta x \Delta u + \Delta y \Delta v}{L_0^2}$$

Engineering/Cauchy strain - not the only choice!

Element displacement vector:

$$\{d^e\} = \{u_i \ v_i \ u_j \ v_j\}^T$$

$$\Rightarrow \varepsilon = \frac{1}{L_0^2} \{-\Delta x \ -\Delta y \ \Delta x \ \Delta y\} \{d\} = \{B_0\}^T \{d\}$$

Definition of linear strain displacement vector:

$$\{B_0\} = \frac{1}{L_0^2} \{-\Delta x \ -\Delta y \ \Delta x \ \Delta y\}^T$$

Relates only to the initial geometry (NB! Not valid on Day 3!!)

VWP – truss analysis

$$\sum_e A^e L_0^e \delta \varepsilon \sigma = \{\delta D\}^T \{P\}$$

Strain-displacement relation

$$\delta \varepsilon = \{B_0\}^T \{\delta d\}$$

Hookes' law (one dimension):

$$\sigma = E \varepsilon = E \{B_0\}^T \{d\}$$

Insert in LHS of VWP:

$$\begin{aligned} \sum_e A^e L_0^e \delta \varepsilon \sigma &= \sum_e A^e L_0^e \{B_0\}^T \{\delta d\} E^e \{B_0\}^T \{d\} \\ &= \sum_e \{\delta d\}^T \left[E^e A^e L_0^e \{B_0\} \{B_0\}^T \right] \{d\} \\ &= \sum_e \{\delta d\}^T [k^e] \{d\} = \{\delta D\}^T \left(\sum_e [k^e] \right) \{D\} \\ &= \{\delta D\}^T [K] \{D\} \end{aligned}$$

VWP – truss analysis

$$\sum_e A^e L_0^e \delta \varepsilon \sigma = \{\delta D\}^T \{P\}$$

VWP in discretized form:

$$\{\delta D\}^T [K] \{D\} = \{\delta D\}^T \{P\}, \quad \forall \{\delta D\}$$

⇓

$$[K] \{D\} = \{P\}$$

Local and global stiffness matrices

$$[k^e] = E^e A^e L_0^e \{B_0\} \{B_0\}^T$$

$$[K] = \sum_e [k^e]$$

Element stiffness matrix

Element stiffness matrix for bar element:

$$\begin{aligned} [k^e] &= E^e A^e L_0^e \{B_0\} \{B_0\}^T \\ &= \frac{E^e A^e}{(L_0^e)^3} \begin{bmatrix} \Delta x^2 & \Delta x \Delta y & -\Delta x^2 & -\Delta x \Delta y \\ \Delta x \Delta y & \Delta y^2 & -\Delta x \Delta y & -\Delta y^2 \\ -\Delta x^2 & -\Delta x \Delta y & \Delta x^2 & \Delta x \Delta y \\ -\Delta x \Delta y & -\Delta y^2 & \Delta x \Delta y & \Delta y^2 \end{bmatrix} \end{aligned}$$

*In code use short expression
for clarity and compactness!*

Global stiffness matrix

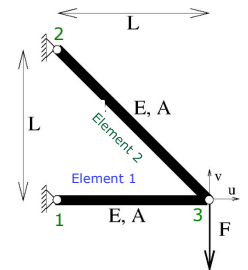
Assembly of element matrices:

$$[k^1] = \begin{bmatrix} k_{11}^1 & k_{12}^1 & k_{13}^1 & k_{14}^1 \\ k_{21}^1 & k_{22}^1 & k_{23}^1 & k_{24}^1 \\ k_{31}^1 & k_{32}^1 & k_{33}^1 & k_{34}^1 \\ k_{41}^1 & k_{42}^1 & k_{43}^1 & k_{44}^1 \end{bmatrix}$$

$$[k^2] = \begin{bmatrix} k_{11}^2 & k_{12}^2 & k_{13}^2 & k_{14}^2 \\ k_{21}^2 & k_{22}^2 & k_{23}^2 & k_{24}^2 \\ k_{31}^2 & k_{32}^2 & k_{33}^2 & k_{34}^2 \\ k_{41}^2 & k_{42}^2 & k_{43}^2 & k_{44}^2 \end{bmatrix}$$

$$[K] = \sum_e [k^e] = [k^1] + [k^2] =$$

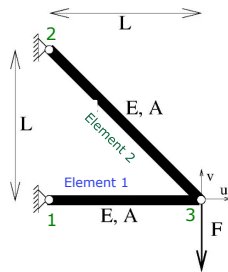
$$\begin{bmatrix} k_{11}^1 & k_{12}^1 & 0 & 0 & k_{13}^1 & k_{14}^1 \\ k_{21}^1 & k_{22}^1 & 0 & 0 & k_{23}^1 & k_{24}^1 \\ 0 & 0 & k_{11}^2 & k_{12}^2 & k_{13}^2 & k_{14}^2 \\ 0 & 0 & k_{21}^2 & k_{22}^2 & k_{23}^2 & k_{24}^2 \\ k_{31}^1 & k_{32}^1 & k_{31}^2 & k_{32}^2 & k_{33}^1 + k_{33}^2 & k_{34}^1 + k_{34}^2 \\ k_{41}^1 & k_{42}^1 & k_{41}^2 & k_{42}^2 & k_{43}^1 + k_{43}^2 & k_{44}^1 + k_{44}^2 \end{bmatrix}$$



Imposing Boundary conditions

Solve FE problem: $[K]\{D\} = \{P\}$

- Elimination of equations
- **Method of 0's and 1's**
- **Penalty method**
- Lagrangian multipliers



Enforcing BC's with method of 0's and 1's

Enforce $D_2 = 0$

$$\begin{bmatrix} k_{11}^1 & 0 & 0 & 0 & k_{13}^1 & k_{14}^1 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & k_{11}^2 & k_{12}^2 & k_{13}^2 & k_{14}^2 \\ 0 & 0 & k_{21}^2 & k_{22}^2 & k_{23}^2 & k_{24}^2 \\ k_{31}^1 & 0 & k_{31}^2 & k_{32}^2 & k_{33}^1 + k_{33}^2 & k_{34}^1 + k_{34}^2 \\ k_{41}^1 & 0 & k_{41}^2 & k_{42}^2 & k_{43}^1 + k_{43}^2 & k_{44}^1 + k_{44}^2 \end{bmatrix} \begin{Bmatrix} D_1 \\ D_2 \\ D_3 \\ D_4 \\ D_5 \\ D_6 \end{Bmatrix} = \begin{Bmatrix} P_1 \\ 0 \\ P_3 \\ P_4 \\ P_5 \\ P_6 \end{Bmatrix}$$

Enforce $D_1 = D_2 = D_3 = D_4 = 0$

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & k_{33}^1 + k_{33}^2 & k_{34}^1 + k_{34}^2 \\ 0 & 0 & 0 & 0 & k_{43}^1 + k_{43}^2 & k_{44}^1 + k_{44}^2 \end{bmatrix} \begin{Bmatrix} D_1 \\ D_2 \\ D_3 \\ D_4 \\ D_5 \\ D_6 \end{Bmatrix} = \begin{Bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ P_5 \\ P_6 \end{Bmatrix}$$

Enforcing BC's with non-zero displacements

Enforce $D_2 = D_2^*$

$$\begin{bmatrix} k_{11}^1 & k_{12}^1 & 0 & 0 & k_{13}^1 & k_{14}^1 \\ k_{21}^1 & k_{22}^1 & 0 & 0 & k_{23}^1 & k_{24}^1 \\ 0 & 0 & k_{11}^2 & k_{12}^2 & k_{13}^2 & k_{14}^2 \\ 0 & 0 & k_{21}^2 & k_{22}^2 & k_{23}^2 & k_{24}^2 \\ k_{31}^1 & k_{32}^1 & k_{31}^2 & k_{32}^2 & k_{33}^1 + k_{33}^2 & k_{34}^1 + k_{34}^2 \\ k_{41}^1 & k_{42}^1 & k_{41}^2 & k_{42}^2 & k_{43}^1 + k_{43}^2 & k_{44}^1 + k_{44}^2 \end{bmatrix} \begin{Bmatrix} D_1 \\ D_2 \\ D_3 \\ D_4 \\ D_5 \\ D_6 \end{Bmatrix} = \begin{Bmatrix} P_1 \\ P_2 \\ P_3 \\ P_4 \\ P_5 \\ P_6 \end{Bmatrix} - D_2^* \begin{Bmatrix} k_{12}^1 \\ k_{22}^1 \\ 0 \\ 0 \\ k_{32}^1 \\ k_{42}^1 \end{Bmatrix}$$

$$\begin{bmatrix} k_{11}^1 & 0 & 0 & 0 & k_{13}^1 & k_{14}^1 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & k_{11}^2 & k_{12}^2 & k_{13}^2 & k_{14}^2 \\ 0 & 0 & k_{21}^2 & k_{22}^2 & k_{23}^2 & k_{24}^2 \\ k_{31}^1 & 0 & k_{31}^2 & k_{32}^2 & k_{33}^1 + k_{33}^2 & k_{34}^1 + k_{34}^2 \\ k_{41}^1 & 0 & k_{41}^2 & k_{42}^2 & k_{43}^1 + k_{43}^2 & k_{44}^1 + k_{44}^2 \end{bmatrix} \begin{Bmatrix} D_1 \\ D_2 \\ D_3 \\ D_4 \\ D_5 \\ D_6 \end{Bmatrix} = \begin{Bmatrix} P_1 - D_2^* k_{12}^1 \\ D_2^* \\ P_3 \\ P_4 \\ P_5 - D_2^* k_{32}^1 \\ P_6 - D_2^* k_{42}^1 \end{Bmatrix}$$

see Cook et al. 13.2 for further details

Enforcing BC's with penalty method

$$\begin{bmatrix} K_{11} & K_{12} & K_{13} & K_{14} & K_{15} & K_{16} \\ K_{21} & K_{22} & K_{23} & K_{24} & K_{25} & K_{26} \\ K_{31} & K_{32} & K_{33} & K_{34} & K_{35} & K_{36} \\ K_{41} & K_{42} & K_{43} & K_{44} & K_{45} & K_{46} \\ K_{51} & K_{52} & K_{53} & K_{54} & K_{55} & K_{56} \\ K_{61} & K_{62} & K_{63} & K_{64} & K_{65} & K_{66} \end{bmatrix} \begin{Bmatrix} D_1 \\ D_2 \\ D_3 \\ D_4 \\ D_5 \\ D_6 \end{Bmatrix} = \begin{Bmatrix} P_1 \\ P_2 \\ P_3 \\ P_4 \\ P_5 \\ P_6 \end{Bmatrix}$$

Enforce $D_1 = D_4 = 0$

$$\begin{bmatrix} K_{11} + \infty & K_{12} & K_{13} & K_{14} & K_{15} & K_{16} \\ K_{21} & K_{22} & K_{23} & K_{24} & K_{25} & K_{26} \\ K_{31} & K_{32} & K_{33} & K_{34} & K_{35} & K_{36} \\ K_{41} & K_{42} & K_{43} & K_{44} + \infty & K_{45} & K_{46} \\ K_{51} & K_{52} & K_{53} & K_{54} & K_{55} & K_{56} \\ K_{61} & K_{62} & K_{63} & K_{64} & K_{65} & K_{66} \end{bmatrix} \begin{Bmatrix} D_1 \\ D_2 \\ D_3 \\ D_4 \\ D_5 \\ D_6 \end{Bmatrix} = \begin{Bmatrix} P_1 \\ P_2 \\ P_3 \\ P_4 \\ P_5 \\ P_6 \end{Bmatrix}$$

In practice: $\infty = \text{large number (e.g. } 10^8 \text{ times max}(K_{ij}))$

Easy to extend to $D_i \neq 0$

Solving the linear equations

Getting displacements:

$$[K]_{corr}\{D\} = \{P\}_{corr}$$

Solution methods:

- Gauss-elimination using factorization
- Iterative methods

Matlab: `D = K_corr \ P`

NumPy: `D = np.linalg.solve(K_corr, P)`

SciPy: `D = sp.sparse.linalg.spsolve(K_corr, P)`

Derived quantities (post-processing)

Element strain: $\varepsilon = \{B_0\}^T \{d\}$

Element stress: $\sigma = E\varepsilon$

Element forces: $N = \sigma A$

Reaction forces:

$$\{R\}_{reaction} = [K]\{D\} - \{P\} = \left[\sum_e \{B_o\} N^e L_0^e \right] - \{P\}$$

Uncorrected !
Cheaper !

General FEM procedure

- 0) Discretize domain into a finite # of elements
(many ways – the more elements the better)
- 1) Node and element numbering
- 2) Build load vector $\{P\}$
- 3) Calculate $[k^e]$ and assemble $[K]$
- 4) Enforce BC's
- 5) Solve for $\{D\}$
- 6) Compute $\varepsilon, \sigma, N \dots$
(interpolation of solutions between nodes) $\rightarrow u$
- 7) Plot and evaluate results

Today

- Download Matlab template^o + example files from DTU Learn
- Add basic FEM procedure to fea.m – remember always to keep copies of things that work!
- Work on exercises 1.1-1.3 (see them as guidelines for what kind of questions you will get in the first assignment).
- Do competition 1 (obligatory, hand-in Monday midnight!)
- Work on *-exercises 1.4-1.6 (voluntary)

Being able to solve standard problems defined in (c.f. 1.1-1.3 for Day 1) for complex and realistic structures and report and discuss the results perfectly is sufficient to get top grades

^o If you have Python experience and prefer that from Matlab, you may also use Python in Weeks 1-4
*-exercises are for fun, further insight and may help in grade decision

Exercises

Edit the example1.m file such that you can model the 9-bar truss structure shown in Figure 1.3.

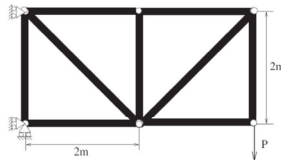


Figure 1.3: A 9-bar truss structure.

Modify and expand the subroutines in "fea" to find displacements, strains and stresses for the truss structures in Fig. 1.2 and Fig. 1.3.

Steps in the program development:

■ ■ ■

Check the results obtained by running the example from Fig. 1.3 with a simple hand-calculation for support reactions and element forces (calculate analytical element forces in at least 3 elements). **Hints:** In the FE-program, the support reactions may be found by calculating the residual (1.10) without inserting boundary conditions.

Exercises

Exercise 1.2

Use the interactive preprocessor program "FIExtract" (download it from the course web-page) to generate a large, more complex truss structure. Draw elements in tension with blue, elements in compression with red and non-loaded elements with green.

Exercise 1.3

Implement prescribed non-zero displacements (see Sec. 2.7 of Cook et al. 2002) and test the code on an illustrative example that demonstrates its use.

Handout Code

```
function fea()
close all
clc

%--- Input file -----
example1 % Input file
%test1 % Input file

neqn = size(X,1)*size(X,2); % Number of equations
ne = size(IX,1); % Number of elements
disp(['Number of DOF ' sprintf('%d',neqn) ...
      ' Number of elements ' sprintf('%d',ne)]);

%--- Initialize arrays -----
Kmatr=zeros(neqn,neqn); % Stiffness matrix
F=zeros(neqn,1); % Force vector
D=zeros(neqn,1); % Displacement vector
R=zeros(neqn,1); % Residual vector
strain=zeros(ne,1); % Element strain vector
stress=zeros(ne,1); % Element stress vector

%--- Calculate displacements -----
[F]=buildload(X,IX,ne,P,loads,mprop); % Build global load vector
[Kmatr]=buildstiff(X,IX,ne,mprop,Kmatr); % Build global stiffness matrix
[Kmatr,R]=enforce(Kmatr,P,bound); % Enforce boundary conditions
% Solve system of equations
[strain, stress]=recover(mprop,X,IX,D,ne,strain, stress); % Calculate element
% stress and strain

%--- Plot results -----
PlotStructure(X,IX,ne,neqn,bound,loads,D, stress); % Plot structure

return
```

- 0) Discretize domain into a finite # of elements
(many ways – the more elements the better)
- 1) Node and element numbering
- 2) Build load vector $\{P\}$
- 3) Calculate $[k^e]$ and assemble $[K]$
- 4) Enforce BC's
- 5) Solve for $\{D\}$
- 6) Compute ϵ , σ , $N \dots$
(interpolation of solutions between nodes) $\rightarrow u$
- 7) Plot and evaluate results

Initial input file

```
% File example1.m, DAY1, modified 3/9 by OS
%
% Example: 3-bar truss
% No. of Nodes: 3
% No. of Elements : 3
clear all

% Coordinates of 3 nodes,
X = [ 0.00 0.00
      0.00 1.00
      1.00 0.00 ];

% Topology matrix IX(node1,node2,propno) ,
IX = [ 1 3 1
       1 2 1
       2 3 1 ];

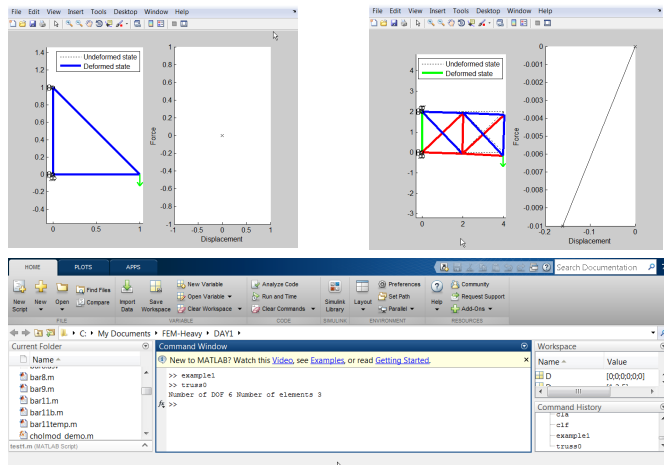
% Element property matrix mprop = [ E A ],
mprop = [ 1.0 1.0 ];

% Prescribed loads mat(node,dof,force)
loads = [ 3 2 -0.01 ];

% Boundary conditions mat(node,dof,disp)
bound = [ 1 1 0.0
          1 2 0.0
          2 1 0.0 ];

% Control Parameters
plotdof = 6;
```

Today's goal!



Solving the linear equations

Getting displacements:

$$[K]_{corr}\{D\} = \{P\}_{corr}$$

Solution methods:

- Gauss-elimination using factorization
- Iterative methods

Matlab: $D = K_corr \backslash P$

NumPy: $D = np.linalg.solve(K_corr, P)$

SciPy: $D = sp.sparse.linalg.spsolve(K_corr, P)$

Derived quantities (post-processing)

Element strain: $\varepsilon = \{B_0\}^T \{d\}$

Element stress: $\sigma = E\varepsilon$

Element forces: $N = \sigma A$

Reaction forces:

$$\{R\}_{reaction} = [K]\{D\} - \{P\} = \left[\sum_e \{B_0\} N^e L_0^e \right] - \{P\}$$

Cheaper !

Uncorrected !

General FEM procedure

- 0) Discretize domain into a finite # of elements
(many ways – the more elements the better)
- 1) Node and element numbering
- 2) Build load vector $\{P\}$
- 3) Calculate $[k^e]$ and assemble $[K]$
- 4) Enforce BC's
- 5) Solve for $\{D\}$
- 6) Compute $\varepsilon, \sigma, N \dots$
(interpolation of solutions between nodes) $\rightarrow u$
- 7) Plot and evaluate results

Today

- Download Matlab template^o + example files from DTU Learn
- Add basic FEM procedure to fea.m – remember always to keep copies of things that work!
- Work on exercises 1.1-1.3 (see them as guidelines for what kind of questions you will get in the first assignment).
- Do competition 1 (obligatory, hand-in Monday midnight!)
- Work on *-exercises 1.4-1.6 (voluntary)

Being able to solve standard problems defined in (c.f. 1.1-1.3 for Day 1) for complex and realistic structures and report and discuss the results perfectly is sufficient to get top grades

^o If you have Python experience and prefer that from Matlab, you may also use Python in Weeks 1-4
*-exercises are for fun, further insight and may help in grade decision

Exercises

Edit the example1.m file such that you can model the 9-bar truss structure shown in Figure 1.3.

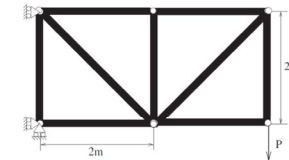


Figure 1.3: A 9-bar truss structure.

Modify and expand the subroutines in "fea" to find displacements, strains and stresses for the truss structures in Fig. 1.2 and Fig. 1.3.

Steps in the program development:

■ ■ ■

Check the results obtained by running the example from Fig. 1.3 with a simple hand-calculation for support reactions and element forces (calculate analytical element forces in at least 3 elements). **Hints:** In the FE-program, the support reactions may be found by calculating the residual (1.10) without inserting boundary conditions.

Exercises

Exercise 1.2

Use the interactive preprocessor program "FIExtract" (download it from the course web-page) to generate a large, more complex truss structure. Draw elements in tension with blue, elements in compression with red and non-loaded elements with green.

Exercise 1.3

Implement prescribed non-zero displacements (see Sec. 2.7 of Cook et al. 2002) and *test the code on an illustrative example that demonstrates its use.*

Handout Code

```
function fea()
close all
clc

%--- Input file ---%
example1 % Input file
%test1 % Input file

neqn = size(X,1)*size(X,2); % Number of equations
ne = size(IX,1); % Number of elements
disp(['Number of DOF ' sprintf('%d',neqn) ...
      ' Number of elements ' sprintf('%d',ne)]);

%--- Initialize arrays ---%
Kmat=zeros(neqn,neqn); % Stiffness matrix
F=zeros(neqn,1); % Force vector
D=zeros(neqn,1); % Displacement vector
R=zeros(neqn,1); % Residual vector
strain=zeros(ne,1); % Element strain vector
stress=zeros(ne,1); % Element stress vector

%--- Calculate displacements ---%
(P)=buildload(X,IX,ne,P,loads,mprop); % Build global load vector
(Kmat)=buildstiff(X,IX,ne,mprop,Kmat); % Build global stiffness matrix
(Kmat,D)=enforce(Kmat,P,bound); % Enforce boundary conditions
% Solve system of equations
[strain, stress]=recover(mprop,X,IX,D,ne,strain, stress); % Calculate element
% stress and strain

%--- Plot results ---%
PlotStructure(X,IX,ne,neqn,bound,loads,D, stress) % Plot structure

return
```

- 0) Discretize domain into a finite # of elements (many ways – the more elements the better)
- 1) Node and element numbering
- 2) Build load vector $\{P\}$
- 3) Calculate $[k^e]$ and assemble $[K]$
- 4) Enforce BC's
- 5) Solve for $\{D\}$
- 6) Compute $\epsilon, \sigma, N \dots$ (interpolation of solutions between nodes) $\rightarrow u$
- 7) Plot and evaluate results

Initial input file

```
% File example1.m, DAY1, modified 3/9 by OS
% Example: 3-bar truss
% No. of Nodes: 3
% No. of Elements : 3
clear all

% Coordinates of 3 nodes,
X = [ 0.00 0.00
      0.00 1.00
      1.00 0.00 ];

% Topology matrix IX(node1,node2,propno),
IX = [ 1 3 1
       1 2 1
       2 3 1 ];

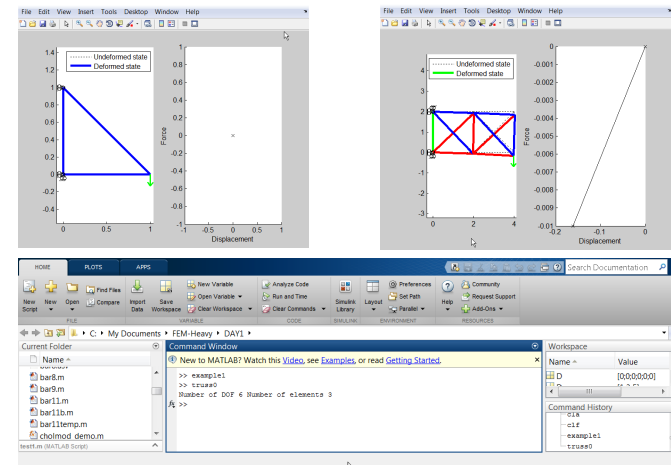
% Element property matrix mprop = [ E A ],
mprop = [ 1.0 1.0 ];

% Prescribed loads mat(node,dof,force)
loads = [ 3 2 -0.01 ];

% Boundary conditions mat(node,dof,disp)
bound = [ 1 1 0.0
          1 2 0.0
          2 1 0.0 ];

% Control Parameters
plotdof = 6;
```

Today's goal!

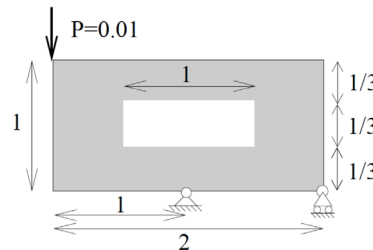


Competition 1

Here is a competition: Who can make the stiffest truss structure? This corresponds to the truss structure with the smallest displacement (in the direction of the load) where the load is applied! ¹

Rules: The truss structure should fit into the grey domain shown below and with no bars crossing the central void area (centered white region). Position and size of the vertical load is fixed (load is applied at the upper left corner). The center of the lower edge is supported in both x and y direction and the lower right corner only in the y-direction. The bar areas and Young's moduli are fixed to $A = 1$ and $E = 1$, respectively. The maximum total length of bar elements is 10 and you may use as many nodes and elements as you want. The stiffness matrix has to be positive definite.

Create an input file called "group#.m" (where # is your group number) and upload it to Learn, September 7th (by midnight). The teachers will evaluate the proposals and announce the prize-winner in the next lecture!

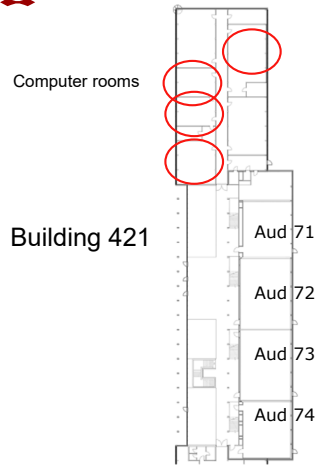


¹The submission (not the ranking!) is part of the evaluation of the first report in the course!

Groups

- Form groups of two:
 - Team up with partner with similar time availability and ambitions
 - If no programming experience: find partner who has !
 - If no solid mechanics experience: find partner who has !
 - Only go for Python if both members have experience in it

Computer exercises



- Distribute in computer rooms
- Organize groups
- Read notes/exercises
- Use notes, book, brain, other students – then teachers / TAs ...

Good luck and have fun!!